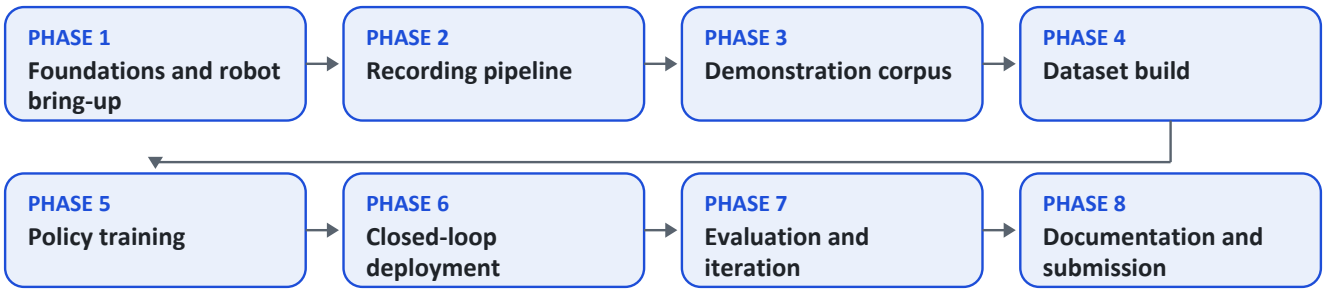
AI Project: work phases

Project 6: imitation learning with a Niryo Ned2, pushing a puck along an unseen path

Team M: Emranul Hoque and Marshal Sebastine. Supervisor: Prof. Dr.-Ing. Thomas Nierhoff. OTH Amberg-Weiden, winter semester 2026/27.

What we do, in what order, and how we know each phase is finished. Phases are ordered by what depends on what, not by dates.



Phase	Goal	Done when
1 Foundations and robot bring-up	Frame the research question, set up the repository and documentation, and get robot, camera, pusher and workspace working from Python with every key number measured.	check_hardware.py passes from a cold boot, config.py holds the measured numbers, six true-scale sheets are taped in place, and the framing, literature notes and contribution log exist.
2 Recording pipeline	One command records a synchronised demonstration episode (video plus robot state) at a rate well above the control rate, and the robot can replay it.	A five-episode pilot on path_01 passes the sync check, is backed up, and one replayed episode reaches the goal; the action representation and control period are written in the decision record.
3 Demonstration corpus	Collect a raw corpus with enough path, pose and recovery variety to generalise, with validation paths held out and test paths never demonstrated.	Every training and validation path has its target episode count, every session passed check_session.py, the test paths have zero episodes, and the raw data is on two disks.
4 Dataset build	Turn raw sessions into a versioned, rebuildable processed dataset with a manifest and the split defined in dataset_config.py.	build_dataset.py is reproducible, the leakage checks pass, the previews look right, and dataset.py loads batches with and without augmentation.
5 Policy training	Train a baseline and one advanced visuomotor policy on the GPU pool, validate them offline on held-out paths, and export the best within the inference budget.	Both policy classes beat the trivial baselines on validation paths, every run is in results_table.md, and the exported policy fits the inference budget on the laptop.
6 Closed-loop deployment	Drive the Ned2 from the exported policy in real time through the single PyNiryo client, inside a safety envelope.	The policy completes path_01 and path_02 from the standard start in at least 4 of 5 trials, every trial is logged, and the loop time stays within the control period.

Phase	Goal	Done when
7 Evaluation and iteration	Measure success on the test paths under a fixed protocol, iterate on data first and model second, and freeze a rehearsed final policy.	summary.csv has at least 10 trials per test path, the failure taxonomy is written up, and the freeze run folder holds the rehearsed configuration and its trials.
8 Documentation and submission	Assemble the documentation and presentation from the section drafts, package the submission, and close the course obligations.	The documentation PDF is within the page limit, the slides fit 15 minutes with both members speaking, the package reproduces from a clean environment, and the question tracker has no open item.

How to use this plan

- A phase starts only when its entry condition exists and ends only when its done-when test passes.
- Tick tasks as they are completed; the outputs name the file or folder in the project layout where each result lives.
- The standing duties on the next page apply during every phase.
- Numbers marked provisional are revised when the professor answers the cited question.

IN FORCE DURING EVERY PHASE

Standing duties

These do not belong to one phase. They run alongside all of them, and skipping one costs points or data.

- [ ] Exam registration: both members register the exam in PRIMUSS within the published window 09.12.26–17.12.26; after registering, confirm the status in PRIMUSS and keep the receipt; a missed registration means no grade.
- [ ] Meetings: attend all four BBB meetings and write one note per meeting, 05_meetings/<yyyy-mm-dd>_meeting_N.md, with results, problems, next steps with an owner each, and the list of runs to keep.
- [ ] Questions: before each meeting copy that meeting's open items from 05_meetings/requests_to_professor.md into the note; afterwards record each answer in the tracker, set its status to answered and cite the note it came from.
- [ ] Moodle: run the README rule 6 sync at its stated cadence (as-of line and changelog in moodle_snapshot.md, a new dated export, new downloads filed in 01_course with a row in the file-name map) and re-read the Moodle FAQ before each meeting.
- [ ] Contribution log: every piece of work gets one line in 05_meetings/contribution_log.md (owner, what, which file, session or run), and every commit is made under the author's own name.
- [ ] Backup: each new raw session is copied to the second disk named in config.py the same day with backup.py and verified (README rule 3); each run's evaluation/ folder and the freeze run are copied the same way after every robot-trial session.
- [ ] Safety during any robot motion, whether scripted, replayed or policy-driven: nobody inside the reach circle, one person's hand on the long-press stop of the top button, the velocity cap set; the stop levels are the software stop key, the long-press of the top button, and the e-stop only for danger because the arm drops.
- [ ] Contribution balance for the 50 % rule: at each meeting count contribution-log lines, episodes recorded, commits and authored sections per member, and rebalance the next work split if one side is clearly behind.
- [ ] Run deletion (README rule 5): after each meeting, delete runs that are cited neither in a meeting note nor in 06_deliverables; the evaluation/ recordings in a run are real robot trials, so they are kept.
- [ ] Read-only inputs (README rules 3 and 4): nothing in 01_course or 03_data/raw is edited, renamed or deleted, except appending to a session's session_notes.md; every episode and trial file starts with its path_id; a bad episode is noted in session_notes.md and excluded in dataset_config.py, never by moving files.

Questions for the professor cited in this plan

Numbering follows 05_meetings/requests_to_professor.md.

Q1: original path-sheet files (SVG or PDF) for true-scale reprints and the line geometry
Q2: which end-effector camera (Niryo's own or a USB webcam) and where it is mounted
Q3: may we print our own path sheets for training, validation and evaluation
Q4: demonstration method (learning mode or jogging) and the pusher tool or pose
Q5: robot software version and permission to update it
Q6: SSH access to the GPU pool
Q7: whether the 7-page documentation with literature review applies, and the submission contents and format
Q8: how the unseen-path test is scored
Q9: test set-up (sheet position relative to the base, puck start, lighting, table)
Q10: offline or local policy, and whether pretrained vision models or LeRobot may be used
Q11: presentation format and whether there is a live robot demo
Q12: final-test logistics (where the test is run and how the test path is handed over)

PHASE 1 OF 8

Foundations: framing, project rails and robot bring-up

Goal	The project is framed as a gradable research question with contribution tracking from the first session, the repository and documentation scaffolding exist in the README layout, physical path sheets at true scale are in hand, and the Ned2, the camera, the pusher, the overhead rig and the laptop work together from Python with every number later phases depend on (software version, call latency, camera rate, smallest smooth step, workspace box) measured and written into config.py.
Starts when	The kit (Ned2 with safety box and e-stop, Custom Gripper, Vision Set camera or none, puck) and the laptop are in hand; README.md, the requirements handout and 05_meetings/requests_to_professor.md exist and both members have read them; the six paths exist only as photos in 01_course/path_sheets, so Q1 (path-sheet files) is asked at meeting 1 and blocks 1.5 until sheets at true scale exist.
Done when	From a cold boot, check_hardware.py calibrates, caps velocity, moves to the home pose, prints joint state, the call latencies for the chosen link, the smallest smooth step and its median blocking time, saves a frame showing puck and line at pushing height and reports camera fps and latency while the robot is moving; config.py holds the latencies, smallest smooth step, camera settings, the measured workspace numbers and the second-disk path; requirements.txt pins the PyNiryo release that matches the recorded software version; six sheets at true scale lie taped at the marked position with the overhead rig photographed and measured in 03_system.md; the dummy torch job has run on the GPU pool or the laptop fallback is written down; 01_introduction.md, 03_system.md, literature_notes.md with the five core notes (DAgger, ACT, Diffusion Policy, LeRobot and the pushing paper), contribution_log.md with entries from both members and a repository with commits from both exist.

1.1 Framing, rails and contribution tracking

- [ ] Write the research question in one sentence (can a visuomotor policy trained by imitation on our own demonstrations over several path shapes push the puck to the goal along a shape it has never seen?) plus two or three testable hypotheses (path variety, recovery data, policy class or chunk length) and the interim metric (goal reached within one puck radius yes or no, fraction of line covered, maximum deviation from the line), marked provisional until Q8 (scoring) and Q9 (test set-up) are answered.
- [ ] Fix the scope boundary in the same file: Niryo tooling only, no hardware or low-level limit changes, no hand-written controller inside the final policy; list the design choices the brief leaves open (action representation, control period, observation, chunk length) each with a default and the question it depends on.
- [ ] Decide the citation style and the final template (LaTeX or Word) now; every section draft stays Markdown with its citations in references.bib, and 8.1 converts the drafts with pandoc.
- [ ] Add the new entries to the README folder map before creating them (rule 2): the root .gitignore, 06_deliverables/team_m_project6_documentation/ (section drafts, figures/, references.bib), 05_meetings/contribution_log.md, 04_runs/results_table.md in the 04_runs row, sync.json in the raw-session row, and path_NN_<shape>.svg beside the .jpeg in the path_sheets row as the home of own-sheet source files.
- [ ] Make AI_Project the git repository root with a root .gitignore for 03_data/, 04_runs/ and the PDFs in 01_course, each member committing under their own name; requirements.txt waits for 1.3 because the PyNiryo pin depends on the robot software version.
- [ ] Start contribution_log.md with the columns of the standing contribution-log duty and split the work into two strands with a named owner each (strand A robot interface, camera, recorder and deployment loop; strand B dataset pipeline, training and offline evaluation), with demonstrations, robot trials and writing shared and each member owning the literature for their strand.
- [ ] Pin 02_code/requirements_train.txt (torch, torchvision, LeRobot if used) and test the compute chain once: OTH VPN, ssh to the pool host named in gpu_pool_ssh_guide_en.pdf, nvidia-smi, create the pool training environment from requirements_train.txt (5.1 reuses it), time a 1 GB transfer and run a dummy torch job (one matrix multiply on the GPU); if access is not active (Q6, GPU access), write down the laptop CPU fallback (small model, reduced image size) and keep the question open.

Outputs: `README.md (folder-map rows for .gitignore, the documentation folder, the contribution log, results_table.md, sync.json and own-sheet .svg sources)`, `.gitignore (repository root)`, `02_code/requirements_train.txt`, `05_meetings/contribution_log.md`, `06_deliverables/team_m_project6_documentation/01_introduction.md (research question, hypotheses, interim metric, design-choice defaults, scope boundary, template decision)`

1.2 Literature notes and method shortlist

- [ ] Build the reading list around the research question with a stopping rule (a paper is in only if it changes a design decision or will be cited), starting from five core papers: DAgger (arXiv 1011.0686) for behaviour cloning and covariate shift, ACT (arXiv 2304.13705), Diffusion Policy (its Push-T task is the closest analogue), LeRobot (dataset format and reference implementations), and one paper on non-prehensile pushing or visual line following.
- [ ] Write one structured note per paper in literature_notes.md (problem, method, data scale and collection method, hardware, evaluation metric, what we borrow, what does not transfer to a 5–15 fps camera with blocking robot calls) and add its references.bib entry before the note is committed; split the reading by strand so each member's paragraphs are their own.
- [ ] Shortlist the policies: BC with a small CNN or ResNet-18 encoder and a chunked action head as the baseline, and exactly one of ACT or Diffusion Policy as the advanced candidate, with a from-scratch encoder variant kept in case pretrained backbones are disallowed (Q10, offline and pretrained models); the decision record is written in 2.3 once the latencies are measured.

Outputs: `06_deliverables/team_m_project6_documentation/literature_notes.md`, `06_deliverables/team_m_project6_documentation/references.bib`

1.3 Safe placement, power-on, software version and robot environment

- [ ] Place the Ned2 on a firm, flat table larger than its 200 × 200 mm base, away from every edge, with the ~500 mm reach circle clear; wire the safety box with the e-stop between the power supply and the robot; photograph the set-up into figures/.
- [ ] Power on with the back-panel button, wait for the blue ring, connect over direct Ethernet (169.254.200.200) so the laptop's Wi-Fi stays free for the VPN, with the hotspot (10.10.10.10) as the fallback link, install NiryoStudio and run the automatic calibration.
- [ ] Read the robot software version in NiryoStudio and record it in 03_system.md and in the meeting-1 note; if it is below 5.5.0, raise Q5 (software version, may we update) and pin the PyNiryo release that the compatibility table matches to it, otherwise pin PyNiryo 1.2.x; only now write 02_code/requirements.txt (Python 3.11: pyniryo, opencv-python, numpy, pandas).
- [ ] Write the start-up and shutdown checklist (calibrate after every reboot, home pose before power-off, never unplug while running, flashing orange ring means collision, fingers out of the joint pinch points, let hot motors cool before handling the arm) and rehearse the three stop levels of the standing safety duty once.

Outputs: `02_code/requirements.txt`, `06_deliverables/team_m_project6_documentation/03_system.md (software version, start-up and shutdown checklist, set-up measurements; later phases append)`

1.4 Robot control probes

- [ ] Run the official PyNiryo setup check (calibrate, move to zero, learning mode) with the workspace clear, then write 02_code/robot_interface.py around the one-client rule: connect, calibrate_auto, velocity cap, get_joints, get_pose, move_step (a move_pose clamped to the workspace box at fixed z, refused when a collision is flagged or the software stop key was pressed, every refusal logged with its reason), move_joints, learning mode on and off, collision_detected and clear_collision_detected, home and shutdown; the workspace box, pushing height z, tool orientation and home pose are provisional values in config.py here (a wide box over the clear table at a safe z above it, the calibration home pose) that 1.5 task 6 replaces with the measured ones.
- [ ] With 02_code/check_hardware.py time get_joints, get_pose and a 5 mm move_step over direct Ethernet and over hotspot Wi-Fi (100 calls each); keep direct Ethernet (1.3) unless its median move_step is slower by more than 20 %, write the kept link's medians into config.py, since they bound the recording rate (phase 2) and the control period (phases 2 and 6), and record the hotspot medians as the fallback link.
- [ ] Confirm the one-client rule by opening a second PyNiryo connection while the first is open and record what happens in 03_system.md; from this fix the architecture as one robot-owner process for recording and deployment alike.
- [ ] With the velocity cap at its lowest setting, execute a chain of small move_step calls and record the smallest step whose pose stream shows no pause above one sample period; because a sent move cannot be aborted, this step becomes the control-loop unit; record its median blocking time over 100 calls into config.py, which 2.1 uses as the provisional move_step latency.
- [ ] Trigger a collision by driving at the lowest velocity cap into a foam block, clear it from code and switch learning mode on and off from code; verify from one script run: cold boot, calibrate, home pose, learning mode on and off.

Outputs: `02_code/robot_interface.py`, `02_code/check_hardware.py` (robot latencies, one-client check, smallest smooth step; the camera check is added in 1.5 and the sync test in 2.2), `02_code/config.py` (one defaults dictionary for machine-wide settings: robot IP and link, velocity cap, call latencies, smallest smooth step and its median blocking time, second-disk path, and the provisional workspace box, z, tool orientation and home pose that 1.5 replaces; later phases add keys named in their tasks)

1.5 Path sheets, camera, pusher, workspace and overhead rig

- [ ] Confirm that physical sheets of path_01 to path_06 exist at true scale; if not, obtain the originals (Q1, path-sheet files) or print them from the photos scaled to a measured reference (the puck diameter visible in each photo), and write the scale factor into 03_system.md.
- [ ] Identify the camera (Vision Set read with get_img_compressed() at 5–15 fps JPEG, or a USB webcam read with OpenCV in its own thread) and write 02_code/camera_interface.py with one API for both; extend check_hardware.py to grab at least 300 frames and report fps, resolution and glass-to-Python latency against a stopwatch shown on screen.
- [ ] Stream frames while the robot executes moves in the same program; if PyNiryo's blocking calls stall the stream the camera stays in its own thread or process; if the Vision Set gives under about 10 fps or a latency spread above one frame period, ask for a USB webcam (Q2, camera).
- [ ] Fix the mount position and orientation (Q2, camera) so that at pushing height the frame shows the puck and a stretch of line ahead of it; lock exposure and white balance; photograph the mount into figures/ and check that the cable cannot snag at extreme poses; write resolution and training crop into config.py.
- [ ] Decide the pusher as the Custom Gripper jaws (closed stock jaws or 3D-printed jaws, 300 g payload, no new tool; Q4, demonstration method and pusher) and confirm with the puck that it pushes without climbing or wedging at one fixed pushing height.
- [ ] Replace the provisional workspace values from 1.4 in config.py with the measured ones (sheet position taped on the table, pushing height z, fixed tool orientation, puck start pose, home pose, workspace box) and jog the pusher over all six sheets to confirm every path lies inside reach at pushing height under the velocity cap.
- [ ] Fix an overhead phone or camera mount above the sheet with a printed scale reference taped inside its view, photograph the whole set-up with a tape measure in frame so it can be rebuilt identically for every session and the final test, and record the mount position and all measurements in 03_system.md.

Outputs: `02_code/camera_interface.py`, `06_deliverables/team_m_project6_documentation/figures/setup_*.jpg (set-up, mount, overhead rig and workspace photos; later phases add their own figure files)`

WATCH OUT

- Robot software older than 5.5.0 with no permission to update: PyNiryo 1.2.x will not connect; the matching older PyNiryo release is pinned instead and the limitation recorded.
- Vision Set at 5–15 fps JPEG is too slow or jittery for closed loop, or blocking calls stall capture in one thread: a USB webcam through OpenCV in its own thread is the fallback.
- GPU pool not activated (Q6, GPU access): the BC baseline trains on the laptop CPU at low resolution meanwhile.
- Sheets printed from photos are off scale: the scale factor is checked against the puck and the overhead reference, and corrected sheets replace them before the pilot.
- Only one member can run the set-up: both rehearse the 1.3 checklist and the 1.4 script run, and share drift is caught by the standing contribution-balance duty.

PHASE 2 OF 8

Recording pipeline and synchronisation

Goal	One command records one demonstration episode as end-effector video plus robot state on one clock at a rate at least twice the control rate, saved in the raw layout, checked and backed up; the demonstration method and the action representation are chosen by recorded trial and replay; the control period is confirmed with a stated inference budget and an inference gate; and the robot can replay an episode on the same sheet.
Starts when	The phase 1 done_when holds (call latencies, camera fps, the smallest smooth step and the measured workspace are in config.py); Q4 (demonstration method and pusher) answered at meeting 1 or the default (learning mode, gripper jaws) written down in 03_system.md.
Done when	The pilot session exists in 03_data/raw and on the second disk with five episodes that pass check_session.py, whose .csv and .mp4 cover the same time span at or above the target rates; the session's sync.json holds a residual below one camera frame period; replay_episode.py reproduces one pilot episode with the puck reaching the goal within one puck radius; the six trial episodes are in raw, checked, backed up and in the exclusion list; config.py holds the confirmed control period, the inference budget and the replay latency gap; 05_method.md names the baseline, the advanced policy, the confirmed action representation, k, the control period, the inference gate and the period rule.

2.1 Recorder, provisional control period and backup

- [ ] Implement 02_code/record_episode.py: the single PyNiryo client polls get_pose() once per sample in a tight loop with time.monotonic() stamps (get_joints() instead only if 2.4 switches to joint deltas), reads the learning-mode flag once per episode, and supports both demonstration methods (learning mode passive, or jog keys sending move_step calls); the built-in 7 Hz recorder is not used; the camera thread stamps each frame from the same clock.
- [ ] Write each episode as <path_id>_episode_NNN.mp4 plus .csv (timestamp, pose, demonstration method), with the episode number counting up per path_id across all sessions (rule 4), a path_id prompt and start, stop and discard keys.
- [ ] At episode end run an integrity check (frame count against fps times the episode's time span, no gap in state timestamps above two sample periods, non-zero file sizes) and prompt for the outcome (success, fail, note), appended to session_notes.md.
- [ ] Compute the provisional control period from the phase 1 numbers (median camera read plus median blocking move_step for the smallest smooth step plus a 100 ms inference budget), write period and budget into config.py, and let 2.3 confirm them after the method trial.
- [ ] Record one test episode in learning mode, read from its integrity check that the achieved state rate is at least twice the provisional control rate and that the camera fps matches check_hardware.py, then discard it with the discard key so nothing from 2.1 lands in raw; tag the recorder in git so every raw session names the recorder version it came from.
- [ ] Write 02_code/backup.py, which copies a raw session, a run's evaluation/ folder or a freeze run to the second disk named in config.py and verifies file count and sizes; from session_01 in 2.3 on it runs under the standing backup duty.

Outputs: `02_code/record_episode.py`, `02_code/backup.py`

2.2 Synchronisation test, dataset config, session check and protocol

- [ ] Extend check_hardware.py with a sync test: tap the pusher on the table in view of the camera (or step the arm) three times, find each visible event in the frames and the pose jump in the state log, and write the median offset and the residual (spread of the three offsets) to 03_data/raw/<yyyy-mm-dd>_session_NN/sync.json, the only place build_dataset.py reads the offset from.
- [ ] Set the sync gate (a residual above one camera frame period fails the session), enforced only by check_session.py; the sync test runs before the first episode of every session.
- [ ] Write 02_code/dataset_config.py with the build-time rule (robot state is interpolated to each frame's timestamp, never the other way round) and the exclusion list, starting empty.
- [ ] Write 02_code/check_session.py: file pairs present, integrity check per episode, sync.json present and within the sync gate, episode numbering continuous per path_id, per-path counts against the targets in dataset_config.py (skipped while no targets are defined, as for the trial and pilot sessions), an automated visibility check (an episode fails when the dark line-pixel count in the training crop drops below 0.5 % of the crop pixels in any frame, provisional until re-set from the pilot's lowest passing frame and written into dataset_config.py), a trajectory overlay on the sheet photo, speed and episode-length histograms, and a contact sheet sampled every 30 frames per episode.
- [ ] Write the session_notes.md template: path_id, demonstrator, sheet pose, puck start, lighting, pusher height, recorder tag, the per-episode procedure (sheet taped at the mark, puck at the start mark, arm at home, the standing safety duty in force before any motion, record, push to goal, stop, outcome noted) and the list of bad episodes.

Outputs: `02_code/dataset_config.py (created here with the interpolation rule and the exclusion list; later phases add the visibility threshold, path sets, episode targets, pose tolerances, schema and the action-space reason)`, `02_code/check_session.py`, `03_data/raw/<yyyy-mm-dd>_session_NN/sync.json (one per session, written by the sync test)`

2.3 Demonstration-method trial, action representation and decision record

- [ ] Record three pushes per method on path_01_straight_line with record_episode.py after the sync test: hand-guided in learning mode (the arm is passive, one member guides while the other runs the recorder) and jogged (keyboard or gamepad sends move_step calls); score each on pose-stream smoothness (no pause above one sample period), the puck centre within one puck radius of the line in every sampled frame and achieved state rate, and choose the primary method and its fallback.
- [ ] Keep the six trial episodes in raw as session_01 under their normal numbers (path_01_straight_line_episode_001 to 006), run check_session.py and backup.py on the session, list the episodes as excluded in dataset_config.py, and continue the pilot in 2.4 at 007.
- [ ] Propose the action representation: planar end-effector deltas (dx, dy, optionally dyaw) at constant z and fixed tool orientation derived from the pose stream, with joint deltas as the fallback; the pilot replay in 2.4 confirms it once.
- [ ] Confirm the control period from 2.1 by re-measuring the camera read and the blocking move_step for the chosen step, write period and budget into config.py, and state in 05_method.md the inference gate (median forward pass at most 70 ms, 70 % of the 100 ms budget, and median loop of camera read, forward pass and move_step at most the control period) that 5.4 applies, together with the period rule: 5.4 fits the policy inside the budget (image size, encoder size, open-loop chunk), and the period changes only by re-running phases 4 and 5 with a longer period, never silently.
- [ ] Write the method decision record: baseline BC plus the one advanced policy from 1.2, action representation, chunk length k, control period, the ablations that test the hypotheses (path variety, recovery data, augmentation, chunk length), and the rejected options with reasons; update the defaults in 01_introduction.md.

Outputs: `06_deliverables/team_m_project6_documentation/05_method.md (decision record, inference gate, period rule and experiment plan; 5.4 appends the selection rule)`, `03_data/raw/<yyyy-mm-dd>_session_01/ (trial episodes path_01_straight_line_episode_001 to 006, session_notes.md with the method comparison)`

2.4 Replay and pilot

- [ ] Implement 02_code/replay_episode.py: drive the robot through a recorded pose trajectory via robot_interface.move_step at reduced speed under the velocity cap and check that the puck follows the line on the same sheet; compare the measured time per replayed step with the recorded interval and write the gap into config.py as the first latency budget for the control loop.
- [ ] Record a pilot of five episodes on path_01_straight_line (episodes 007 to 011), run check_session.py and backup.py, and replay one pilot episode: if the puck reaches the goal within one puck radius the planar-delta representation is confirmed, otherwise switch the recorder to get_joints(), adopt joint deltas and re-record the pilot; record the confirmation, or the switch to joint deltas, in the 05_method.md decision record and in the 01_introduction.md defaults.
- [ ] Review the check_session.py plots for frame drops, state gaps, camera view and pusher contact, re-set the visibility threshold in dataset_config.py from the pilot's lowest passing frame, and fix the pipeline before phase 3.

Outputs: `02_code/replay_episode.py`, `03_data/raw/<yyyy-mm-dd>_session_NN/path_01_straight_line_episode_007 to 011 (.mp4 and .csv, the pilot)`

WATCH OUT

- PyNiryo calls block and only one client is allowed, so the polling rate may be lower than hoped; the answer is a longer control period with smaller steps and resampling at build time, never a second client.
- Hand-guided pushes are jerky or lift the pusher: replay exposes this early; jogged teleop is the fallback.
- Camera and state clocks drift: without the per-session sync gate the policy learns a lagged action.
- A raw session lost to disk failure or an accidental edit: the standing backup duty plus the standing read-only-inputs duty.

PHASE 3 OF 8

Demonstration corpus: collection and quality

Goal	A raw corpus with enough path variety, sheet-pose variety, recovery behaviour and set-up consistency to generalise to a path the policy has never seen, with validation paths (demonstrated, held out of training) and test paths (never demonstrated, robot-only) declared before recording, every session checked and backed up, bad episodes flagged in notes and nothing in raw moved or edited.
Starts when	The phase 2 done_when holds; Q1 (path-sheet files) and Q3 (own sheets) answered at meeting 1; Q8 (scoring) and Q9 (test set-up) are meeting-2 items, so their assumptions are written into dataset_config.py until answered.
Done when	check_session.py passes for every session; the corpus holds every training and validation path at the planned episode count including recovery episodes from both demonstrators; the test paths have zero episodes in raw; the exclusion list is in dataset_config.py; every session is on two disks and 03_data/raw has not been edited.

3.1 Collection design

- [ ] Declare the three path sets in dataset_config.py before recording, as two cases marked provisional until Q3 (own sheets), Q8 (scoring) and Q9 (test set-up) are answered: with own sheets allowed, test = path_04_meander, path_05_alpha_loop and every own evaluation sheet (never demonstrated), validation = two own sheets (demonstrated, excluded from training), training = path_01_straight_line, path_02_arc, path_03_two_curve, path_06_alpha_loop_dashed and the remaining own sheets; without own sheets, test = path_04_meander, validation = path_05_alpha_loop, training = the other four with sheet-pose variety; path_06 stays in training in both cases because dashed segments must be seen to be recognised.
- [ ] Declare the held-out sheet poses (rotation and offset) for robot evaluation in the same file, each at least 15° or 50 mm from every training pose, together with the sheet-pose tolerance (±2° and ±5 mm count as the same pose), provisional until Q9 (test set-up) is answered.
- [ ] If allowed (Q3, own sheets), design training, validation and evaluation sheets (path_07 onward, numbered in arrival order, the source .svg beside the .jpeg, read-only once added to 01_course/path_sheets) with varied curvature, turns and line widths and print them on the same paper and ink; if not, rotated and shifted originals are the variety source.
- [ ] Define the variation axes per episode (puck start offset, sheet rotation and offset, lighting level, demonstrator, push speed) and write the per-path episode targets into dataset_config.py sized to the available robot time (provisional: 20 episodes per training path, 10 per validation path), with the collection-stop rule: phase 3 collection stops when every training and validation path reaches its target in dataset_config.py; further episodes are recorded only in 7.3 and stop under its plateau rule.
- [ ] Plan recovery demonstrations (start with the puck pushed off the line and demonstrate the correction, the cheap DAgger substitute against compounding error; provisional: one recovery episode in four) and fix the quality rule: an episode counts only if the puck reaches the goal within one puck radius with the line in view throughout, otherwise it is handled under the standing read-only-inputs duty.

Outputs: `01_course/path_sheets/path_07_<shape>.jpeg and .svg onward (own sheets, if Q3, own sheets, allows them)`, `06_deliverables/team_m_project6_documentation/04_dataset.md (protocol, path sets and variation plan; 3.3 appends the audit)`

3.2 Collection sessions with per-session checks

- [ ] Record sessions following the 2.2 protocol with both members demonstrating every training and validation path and never a test path; contribution_log.md records who recorded which episodes.
- [ ] Before recording, compare the session's first frame and overhead photo against the phase 1 reference photos on the overhead scale reference and stop when the sheet corners or the camera mount sit more than 5 mm from the reference or the first frame's mean brightness differs by more than 10 %.
- [ ] Run check_session.py after every session and fix the cause of any failure (set-up drift, glare, occlusion, puck slip) before the next session; bad episodes are handled under the standing read-only-inputs duty.

Outputs: `03_data/raw/<yyyy-mm-dd>_session_NN/ (one per collection session: session_notes.md, sync.json and episode files)`

3.3 Corpus audit

- [ ] Audit the corpus for balance from the check_session.py summaries across all sessions (episodes per path against target, share of recovery episodes, both demonstrators represented, sheet poses covered, exclusion rate per session) and record in 04_dataset.md what made the dataset hard and what was changed, the professor's stated hard part.
- [ ] Confirm from the same summaries that every test path has zero episodes in raw and every validation path has at least its target count.

Outputs: `06_deliverables/team_m_project6_documentation/figures/dataset_*.png (trajectory overlays and histograms from check_session.py)`

WATCH OUT

• Too few distinct paths: the policy memorises shapes and fails the unseen path; variety and recovery episodes beat repeats.
• Set-up drift between sessions (sheet position, camera mount, light) is caught only because check_session.py and the reference-photo comparison run every session.
• Test conditions unknown (Q8, scoring; Q9, test set-up): record under several sheet poses and lighting levels so the test falls inside the training distribution.
• Robot time is finite: the per-path targets in dataset_config.py are sized to it, and collection stops when they are reached (3.1 collection-stop rule).

PHASE 4 OF 8

Dataset build

Goal	A versioned, rebuildable processed dataset of (image, state, action-chunk) samples split by path set, with a manifest and normalisation statistics, loaded directly by the training code; developed on the pilot and re-run as the corpus grows.
Starts when	Phase 2 pilot data and the six trial episodes in 03_data/raw, the confirmed action representation and the control period from phase 2 (the build is developed on the pilot while phase 3 runs).
Done when	Running build_dataset.py twice from the same raw sessions and config yields an identical manifest.json; the sample previews show frames with correctly oriented action arrows; on v01_pilot the declared pilot episode is in the validation split (the one exemption from the path-id rule), and on v<NN>_full_corpus manifest.json lists the validation path_ids in the validation split and the test path_ids nowhere; dataset.py loads a batch with and without augmentation; dataset_config.py records the chunk-timing replay result.

4.1 Schema, chunk timing and build pipeline

- [ ] Fix the sample schema in dataset_config.py: image at the training crop and resolution, optional proprioception (x, y, yaw), and an action chunk of length k of deltas at the control period; the on-disk format is one npz file per episode, provisional until Q10 (offline and pretrained models) is answered, with the migration step stated: if LeRobot is allowed, build_dataset.py gains a writer for LeRobotDataset parquet plus mp4 and the next version is built in that format from the same raw sessions.
- [ ] Write 02_code/build_dataset.py: read raw sessions, apply the exclusion list, read each session's sync.json and interpolate state to frame timestamps, compute action chunks at the control period, compute normalisation statistics on the training split only, and write 03_data/processed/v<NN>_<description>/ with manifest.json (source sessions, recorder tag, exclusions, path sets, code commit, statistics).
- [ ] Split by path set from dataset_config.py: training paths to train, validation paths to validation, test paths absent, never a random frame split; v01_pilot, which holds only path_01, declares one pilot episode as validation in dataset_config.py for the 5.1 chain test only.
- [ ] Replay the chunked actions of one pilot episode through replay_episode.py to check that the dt and the chunk timing reproduce the demonstrated push (puck reaching the goal within one puck radius), and record the result in dataset_config.py next to the action-space entry; the representation itself is not re-opened.
- [ ] Add checks that fail the build: no NaN, action magnitudes at most the velocity cap times the control period, chunks never cross episode boundaries, no episode in two splits, and every validation or test path_id declared in dataset_config.py absent from train; v01_pilot's single validation episode is declared by episode id and is the one exemption from the path-id rule.
- [ ] Render a handful of samples (frame plus action arrow) to png for a visual check, rebuild after every collection session, and name the version built when the phase 3 done_when holds v<NN>_full_corpus.

Outputs: `02_code/build_dataset.py`, `03_data/processed/v01_pilot/manifest.json`, `03_data/processed/v01_pilot/samples/ (png previews)`, `03_data/processed/v<NN>_full_corpus/manifest.json (the first version built from every training and validation path at target count)`

4.2 Loader and augmentation

- [ ] Write 02_code/dataset.py: loads a processed version from its manifest, returns image, state and action chunk, applies the stored normalisation.
- [ ] Add image augmentation behind a flag (colour jitter, small random crop or shift, blur; no flips unless the actions are mirrored to match), off for validation and for the sample viewer.

Outputs: `02_code/dataset.py`

WATCH OUT

• An action timing error (lag or wrong dt) is invisible in loss curves and shows up only as a lagging policy on the robot; the replay of chunked actions in 4.1 is the guard.

• Leakage of validation or test paths into train makes every offline number meaningless; the build check fails on it.

PHASE 5 OF 8

Policy training and offline validation

Goal	A baseline and one advanced visuomotor policy trained on the GPU pool or the laptop fallback, reproducible from dataset version plus code commit, validated offline on the validation paths against trivial baselines, and exported so that the deployment loop passes the inference gate from 2.3.
Starts when	5.1 and the 5.2 sanity checks need only v01_pilot and the 2.3 decision record, so they may run while phase 3 collects; 5.2 task 3, 5.3 and 5.4 need v<NN>_full_corpus (the phase 3 done_when). Also needed: GPU-pool access (Q6, GPU access) or the laptop fallback confirmed, and Q10 (offline and pretrained models) asked at meeting 2, with the from-scratch encoder as the default until answered.
Done when	Both policy classes have at least one completed run whose config.json records dataset version and commit; the BC run beats the constant and nearest-neighbour baselines on every validation path and the overfit and shuffled-label checks pass; results_table.md has one row per run on the same splits; the selected policy_export/ loads on the laptop in deploy_policy.py --benchmark and its latency.json passes the inference gate from 2.3.

5.1 Training infrastructure

- [ ] Write the two transfer commands (02_code and 03_data/processed to the pool; 04_runs back to the laptop over the VPN) into 03_system.md and reuse the pool environment from 1.1; no transfer script.
- [ ] Write 02_code/train.py: takes a config dictionary merged over defaults, writes 04_runs/<yyyy-mm-dd>_<model>_<what_changed>/ with config.json (dataset version, git commit, hyperparameters), checkpoints and logs, and at the end of every run writes policy_export/ (weights, normalisation statistics, preprocessing config, action-space definition) that loads on the laptop outside the training code.
- [ ] Write 02_code/evaluate_offline.py: validation action error per validation path compared with a constant-mean-action baseline and a nearest-neighbour baseline, open-loop rollout error in mm (mean distance between the predicted and the demonstrated trajectory) with its plot, and one row appended to 04_runs/results_table.md per run (run folder, model, dataset version, validation error per path, rollout error, test-path success rate left for phase 7).
- [ ] Run train.py and evaluate_offline.py end to end on v01_pilot as a run-without-error check of the chain (the declared pilot validation episode supplies the validation and rollout numbers) before v<NN>_full_corpus exists.

Outputs: `02_code/train.py`, `02_code/evaluate_offline.py`, `04_runs/results_table.md (the single source of every results table; make_figures.py regenerates 06_experiments.md from it)`

5.2 Baseline behaviour cloning

- [ ] Write 02_code/policy_bc.py: small CNN or ResNet-18 encoder (pretrained only if Q10, offline and pretrained models, allows it, else from scratch) plus an MLP head predicting an action chunk, optional state input.
- [ ] Pipeline sanity before any real run: overfit one v01_pilot episode until the training loss is below 1 % of its starting value, then train with shuffled labels and confirm that the validation error is no better than the constant-mean baseline; both results go into the run's metrics.json.
- [ ] Train on v<NN>_full_corpus with augmentation and record the validation error against both trivial baselines and the rollout error in the run folder and in results_table.md.

Outputs: `02_code/policy_bc.py`, `04_runs/<yyyy-mm-dd>_bc_<what_changed>/ (config.json, metrics.json, rollout plots, policy_export/)`

5.3 Advanced policy and ablations

- [ ] Implement or adapt the one advanced policy from the decision record (ACT or Diffusion Policy, the LeRobot implementation if Q10, offline and pretrained models, allows it) on the same dataset, splits and metrics as the baseline.
- [ ] Run only the ablations listed in the decision record (path variety, recovery data on or off, augmentation, chunk length, proprioception); one change per run, stated in the folder's <what_changed>.

Outputs: `02_code/policy_act.py or 02_code/policy_diffusion.py`, `04_runs/<yyyy-mm-dd>_<model>_<what_changed>/ (one per ablation run here and per iteration run in 7.3)`

5.4 Selection and inference gate

- [ ] Append the selection rule to 05_method.md (validation error on validation paths and rollout error in mm, never training loss) and apply it to pick at most two checkpoints for the robot.
- [ ] Write the first mode of 02_code/deploy_policy.py, --benchmark: load a policy_export/, time one forward pass on the laptop at the deployment image size, one camera read and one blocking move_step separately over 100 iterations, and write the medians and their sum to the run's latency.json.
- [ ] Apply the inference gate from 2.3 to latency.json (median forward pass at most 70 ms, median loop at most the control period); if a candidate fails, reduce image size, shrink the encoder or execute a longer chunk open-loop, and if none fits, the period rule from 2.3 applies.
- [ ] Copy the results table into 06_experiments.md as its first version (make_figures.py regenerates it in 8.1).

Outputs: `04_runs/<run>/latency.json`, `06_deliverables/team_m_project6_documentation/06_experiments.md (generated from results_table.md)`

WATCH OUT

• Good offline numbers with bad on-robot behaviour (covariate shift): offline metrics are a filter, phase 7 decides.
• Pretrained backbones disallowed (Q10, offline and pretrained models): the from-scratch encoder with stronger augmentation is the fallback.
• LeRobot API churn between versions: pin versions in requirements_train.txt.
• Validation paths differ in difficulty from the professor's unseen path: the validation number only ranks runs; the robot result on test paths in phase 7 is the evidence.

PHASE 6 OF 8

Closed-loop deployment

Goal	The exported policy drives the Ned2 in real time through the single PyNiryo client, inside a safety envelope that respects the blocking calls and the no-abort rule, and pushes the puck along a training path end to end.
Starts when	The phase 5 done_when holds (a selected policy_export/ whose latency.json passes the inference gate); robot_interface.py with move_step, the workspace box and the smallest smooth step from phase 1; replay_episode.py from phase 2.
Done when	The policy completes path_01_straight_line and path_02_arc from the standard start in at least 4 of 5 logged trials each without a safety stop; every trial folder contains video, state and action logs, overhead_end.jpg and a stop reason where one occurred; the measured loop period in the logs is at most the control period assumed at build.

6.1 Control loop and watchdog

- [ ] Complete 02_code/deploy_policy.py: one process owns the robot; the camera thread keeps the latest frame with its timestamp; preprocess exactly as dataset.py does; the policy predicts a chunk; the first m actions execute as blocking move_step calls each between the smallest smooth step and the velocity cap times the control period (receding horizon, optional temporal ensembling); then re-observe.
- [ ] Add the watchdog to robot_interface.py: refuse the move and stop the loop when the latest frame is older than one control period, when a robot call exceeds the control period, when collision_detected is set or when the software stop key was pressed, each logged with its reason.
- [ ] Log every trial (video, states, predicted actions, timestamps, stop reason) to 04_runs/<run>/evaluation/<path_id>_trial_NNN/, where the member at the stop button photographs the sheet from the 1.5 overhead rig once the arm has stopped and files it as overhead_end.jpg; the folder is copied under the standing backup duty.

Outputs: `02_code/deploy_policy.py (benchmark mode from 5.4 plus the control loop)`

6.2 Staged dry runs

- [ ] Replay a recorded episode's actions through deploy_policy.py with the policy disabled to validate the action-to-motion mapping and the step timing against the latency budget from 2.4.
- [ ] Shadow mode: the policy prints its actions while a member guides the arm along path_01_straight_line in learning mode; the printed delta heading must lie within 20° of the line tangent in at least 9 of 10 sampled steps.
- [ ] Run the policy with the pusher raised above the puck, then with the puck on path_01_straight_line, then on path_02_arc, under the standing safety duty.

Outputs: `04_runs/<run>/evaluation/<path_id>_trial_NNN/ (one folder per trial: video, state and action logs, overhead_end.jpg, stop reason)`

6.3 Observation matching and tuning

- [ ] Compare live frames with dataset frames (identical resolution, crop and colour order, and a brightness-histogram intersection of at least 0.9 against the dataset's mean histogram), save the comparison figure, and fix mount or lighting drift before touching the policy.
- [ ] Tune step size, executed chunk length m and temporal ensembling under the period rule from 2.3, one change per run, each setting recorded in a deployment section of the run's config.json and the chosen defaults in config.py; keep the same start procedure as in training (sheet taped at the mark, puck at the start, arm at home).
- [ ] Append the per-trial start procedure and stop checklist to 03_system.md.

Outputs: `06_deliverables/team_m_project6_documentation/figures/deploy_*.png (live-frame versus dataset-frame comparison)`

WATCH OUT

• Blocking move() plus inference may cap the loop at a few Hz: small steps and open-loop chunk execution absorb that, which is why the recording rate from phase 2 is at least twice the control rate.
• Latency causes oscillation across the line: temporal ensembling and smaller steps are the first fixes.
• A clamp or stop that triggers too eagerly hides the real behaviour: the logged stop reason tells which.
• A stale frame or a slow robot call makes the policy act on an old observation: the watchdog stops the loop and logs it instead.

PHASE 7 OF 8

Evaluation and iteration

Goal	A measured success rate on the test paths under a fixed protocol, an iteration loop that pulls the data lever first and the model lever second until the result plateaus or the robot-time budget is spent, and a frozen, rehearsed final policy.
Starts when	The phase 6 done_when holds (path_01_straight_line and path_02_arc completed in at least 4 of 5 logged trials each without a safety stop); v<NN>_full_corpus from phase 3 is built; Q8 (scoring) and Q9 (test set-up) answered at meeting 2, or the team's own metric written down as provisional.
Done when	summary.csv reports success rate, path covered and deviation for the final policy on every test path with at least 10 trials each; results_table.md traces every iteration from dataset version to test-path result; the failure taxonomy with counts is in 07_results_discussion.md; the freeze run folder holds the export, config, dataset version and commit, its evaluation/ holds the rehearsal trials, and it is on the second disk; the rehearsal ran from power-on to finish without manual intervention and the video exists.

7.1 Evaluation protocol

- [ ] Define the metrics, provisional until Q8 (scoring) is answered: success (puck centre within one puck radius of the goal mark), fraction of path covered, mean and maximum deviation from the line measured on the trial's overhead_end.jpg from the phase 1 rig against the sheet geometry (the SVG if Q1, path-sheet files, is answered, otherwise the scaled photo), time to goal, number of stops.
- [ ] Fix the trial procedure, provisional until Q9 (test set-up) is answered: at least 10 trials per path per candidate at the held-out sheet poses from 3.1, standard start, test paths only, no retakes, overhead_end.jpg taken as in 6.1 after every trial, one scoring row per trial, both members run trials.
- [ ] Write 02_code/evaluate_robot.py: scores each trial folder from its overhead_end.jpg and logs, appends one row to the run's evaluation/summary.csv, and writes the run's test-path success rate into its results_table.md row.

Outputs: `02_code/evaluate_robot.py`, `04_runs/<run>/evaluation/summary.csv`

7.2 Test-path evaluation and failure taxonomy

- [ ] Run the protocol on every test path for each candidate checkpoint, the same number of trials per candidate.
- [ ] Classify every failed trial (lost the line at a sharp turn, self-crossing, dashed segment, puck slip, drift at the start, latency, safety stop) with counts, and map each category to a fix: more demonstrations at that feature, recovery episodes, augmentation, chunk length, executed chunk length.
- [ ] Pick the model to iterate on from results_table.md.

Outputs: `06_deliverables/team_m_project6_documentation/07_results_discussion.md (failure taxonomy with counts)`

7.3 Iteration loop

- [ ] Data lever first: record targeted episodes for the observed failures with the phase 2 pipeline on training paths (never a test path), rebuild as the next dataset version, retrain the same model, redeploy, re-evaluate under the same protocol.
- [ ] Model lever second: only when more data stops helping, change the policy (chunk length, the advanced class, ensembling), one change per run.
- [ ] Keep one row per iteration in results_table.md (dataset version, run, test-path success rate) and stop when two consecutive iterations change the success rate by no more than one trial in N (the trial-count noise, N being the trials per path) or the robot-time budget is spent.
- [ ] List in each meeting note the runs to keep, the input to the standing run-deletion duty.

Outputs: `03_data/processed/v<NN>_<description>/ (one per iteration after v<NN>_full_corpus)`

7.4 Rehearsal and freeze

- [ ] Freeze the final policy export, deployment config and dataset version in one run folder 04_runs/<yyyy-mm-dd>_<model>_freeze/ with "frozen": true and the git commit in config.json, commit, and write the one-page run instructions a stranger could follow.
- [ ] Rehearse the final test end to end from the freeze folder, on a freshly printed sheet never shown to the policy (filed first as path_NN_<shape>.jpeg plus .svg in 01_course/path_sheets), or on a test path at a held-out sheet pose if printing is not allowed (Q3, own sheets): one attempt, the full power-on, calibrate, camera check, model load, run, shutdown sequence, timed; its trials land in the freeze folder's evaluation/, which is then copied under the standing backup duty; record the rehearsal as the fallback demo video.

Outputs: `04_runs/<yyyy-mm-dd>_<model>_freeze/ (policy_export/, config.json, evaluation/ of the rehearsal)`, `06_deliverables/team_m_project6_demo_video.mp4`, `06_deliverables/team_m_project6_demo_runbook.md (run instructions; 8.2 appends the demo checklist)`

WATCH OUT

- The professor's scoring differs from the team's metric: align as soon as Q8 (scoring) is answered and re-score the logged trials, which every trial's video and overhead_end.jpg make possible.
- Too few trials per path make differences between models noise: the 10-trial minimum and the one-in-N plateau rule are sized for that.
- Late iterations tempt last-minute changes: the freeze protects the rehearsed configuration.

PHASE 8 OF 8

Documentation, presentation and submission

Goal	The 7-page documentation with literature review assembled from the section drafts of phases 1 to 7, the team presentation with a demo plan, a submission package that reproduces the result in a clean environment, and the question tracker closed, with each member's contribution identifiable.
Starts when	Phase 7 results exist (at least a first test-path evaluation); Q7 (documentation and submission contents), Q11 (presentation) and Q12 (final-test logistics) answered or assumed from the module handbook; every non-result section can be finished as soon as its phase is done.
Done when	The documentation PDF is within the page limit, contains the literature review, a conclusion and a contributions section naming who did what, every number in it comes from make_figures.py and matches a run in 04_runs, and every references.bib entry is cited; the slides fit the handbook's time limit in a timed rehearsal with both members speaking; the submission package runs one trial from a clean environment and is uploaded with confirmation; requests_to_professor.md has no open item; both exam-registration receipts and all four meeting notes exist from the standing duties.

8.1 Documentation assembly

- [ ] Fix the outline and page budget (introduction and research question, related work about 1.5 pages, system, dataset, method, experiments, results and discussion, conclusion, individual contributions, references) in the template decided in 1.1.
- [ ] Expand literature_notes.md into 02_related_work.md (imitation learning and covariate shift, ACT, Diffusion Policy with Push-T as the closest analogue, DAgger, LeRobot, pushing tasks), citing each with the decision it shaped; finish every other section (01 to 07) from its draft and write 09_conclusion.md.
- [ ] Write 02_code/make_figures.py so every figure and table comes from 03_data/processed, 04_runs and results_table.md (which it regenerates into 06_experiments.md) and the numbers in the text match the runs.
- [ ] Write 08_contributions.md from contribution_log.md and the git history; tag every section with its author; cross-review the partner's sections against the runs; convert the Markdown sections with pandoc and references.bib into the template decided in 1.1; check page count, every references.bib entry cited and figure numbering; export the PDF.

Outputs: `06_deliverables/team_m_project6_documentation/02_related_work.md`, `06_deliverables/team_m_project6_documentation/08_contributions.md`, `06_deliverables/team_m_project6_documentation/09_conclusion.md`, `02_code/make_figures.py`, `06_deliverables/team_m_project6_documentation/figures/report_*.png (generated by make_figures.py)`, `06_deliverables/team_m_project6_documentation_report.pdf`

8.2 Presentation and demo

- [ ] Build the slides (problem and research question, literature, system, data pipeline, policy, results, failure analysis, contributions, lessons); each slide owned by one member, equal speaking share, each member presents their own strand.
- [ ] Rehearse against the handbook's time limit with the demo video; prepare answers for the expected questions (why this policy, why it fails, what more data would do).
- [ ] Append the demo checklist to the runbook (transport, placement, power and calibration, camera, laptop and model, puck, sheets, the unseen-path procedure, the fallback video ready to play) and dry-run it once under the presentation conditions once Q11 (presentation) and Q12 (final-test logistics) are answered.

Outputs: `06_deliverables/team_m_project6_presentation.pdf`

8.3 Submission package

- [ ] Assemble 06_deliverables/team_m_project6_submission/: code snapshot at the frozen commit with requirements and run instructions, the freeze run's policy_export/, the documentation PDF, the slides, the demo video, the dataset manifest (the dataset itself only if Q7, documentation and submission contents, asks for it) and the contribution statement.
- [ ] Reproducibility check in a clean Python 3.11 environment on the laptop: install, build_dataset.py on one sample session, train.py smoke test, deploy_policy.py loads the export and runs one trial.
- [ ] Freeze at the team's own buffer point, upload where the Moodle submission activity says, verify the upload and keep the confirmation.

Outputs: `06_deliverables/team_m_project6_submission/`

8.4 Closing the question tracker

- [ ] Close requests_to_professor.md: every item answered with the meeting note cited, or marked with the assumption made and where the plan relies on it.
- [ ] Run the final README rule 6 sync and confirm that the package matches the submission contents as answered in Q7 (documentation and submission contents).

Outputs: `05_meetings/requests_to_professor.md (closed)`, `01_course/moodle_snapshot.md (final sync)`

WATCH OUT

- Q7 (documentation and submission contents) may change the documentation scope: writing the sections inside each phase costs nothing either way and is the presentation's backbone.
- Results arrive late: all non-result sections are complete before the final iteration.
- A live demo can fail in front of the audience: the video, the runbook and the results table carry the presentation.